MecHand: freehand drawing of reaction mechanisms with in-browser handwriting recognition and direct export to LaTeX

Jordi Villà-Freixa [1,2,3*]

[1*]Department of Biosciences, University of Vic – Central University of Catalonia (UVic-UCC), Vic, Spain.
[2]IRIS-CC (Institut de Recerca i Innovació en Ciències de la Vida i de la Salut a la Catalunya Central), Vic, Spain.
[3]Computational Biochemistry and Biophysics Lab, Research Group on Bioinformatics and Bioimaging (BI-SQUARE), Universitat de Vic – Universitat Central de Catalunya (UVic-UCC), Vic, Spain.

Corresponding author(s). E-mail(s): jordi.villa@uvic.cat;

Abstract

Curved-arrow reaction mechanisms are central to how chemists reason, teach and communicate, and they are still mostly drawn by hand. Turning such a sketch into a figure for a manuscript means redrawing it in a structure editor, and optical chemical structure recognition (OCSR) tools, which read molecules from images, do not handle the electron-flow arrows that make a drawing a mechanism. We present MecHand, an open-source web application that adds freehand mechanism drawing to the Ketcher chemical editor. Structures, charges and curved arrows are drawn with a stylus, a trackpad or a mouse. An online recogniser that runs entirely in the browser reads the pen strokes, classifies atom labels with a point-cloud gesture recogniser, anchors every curved arrow to an atom or a bond, checks the electron bookkeeping, and inserts the result into Ketcher after the user confirms it. Mechanisms are exported to TikZ and `chemfig` for LaTeX and to CDXML and MDL formats. On synthetic handwriting of thirteen textbook mechanisms from organic, inorganic and biological chemistry, drawn in five hand styles, MecHand recovers 86 % of the atoms, 84 % of the bonds and all curved and reaction arrows. On the 126 drawings of the individual molecules it returns the exact structure for 65 % of them, against 40 % for DECIMER's model for hand-drawn images, with a median recognition time of 0.1 s. The application is tested

automatically in the Chromium, Firefox and WebKit browser engines, including a test that no data leaves the browser.

Scientific contribution: MecHand is, to our knowledge, the first open-source tool that recognises complete hand-drawn reaction mechanisms — structures together with their electron-flow arrows — from pen strokes inside the web browser, checks their electron flow and exports them directly to LaTeX. It shows that online, stroke-based recognition, with a small and openly extensible symbol model, can outperform image-based recognition of hand-drawn molecules in an interactive setting, while keeping every drawing on the user's computer.

Keywords: reaction mechanism, curved arrows, electron flow, sketch recognition, handwriting recognition, chemical structure editor, Ketcher, LaTeX, chemfig, TikZ

1 Background

The curved-arrow notation of reaction mechanisms is one of the most characteristic languages of chemistry. Each arrow states where a pair of electrons (or, with a half-headed "fishhook", a single electron) comes from and where it goes, and a sequence of such arrows explains how bonds are broken and formed. Learning to draw and read these arrows is a recognised difficulty for students [1, 2], and mechanisms are therefore drawn constantly — on paper, on whiteboards and on tablets — in teaching and research.

Turning a sketch into a figure for a paper, a thesis or lecture notes is, however, still a manual task. Commercial structure editors such as ChemDraw and Marvin provide mechanism arrows; Marvin JS attaches its electron-flow arrows to lone pairs, bonds and atoms and enforces rules on valid sources and targets [3]. Open-source editors for the web, such as Ketcher [4, 5], JSME [6] and Kekule.js [7], are widely embedded in databases and teaching platforms. All of them are operated with a mouse and toolbars, so the user has to translate what they would draw by hand into a sequence of tool choices. In documents typeset with LaTeX, which is common in chemistry and physics, structures and arrows are usually rewritten by hand in packages such as `chemfig` [8] or TikZ [9], a slow and error-prone step.

Two lines of work address the gap between hand-drawn and digital chemistry. *Optical chemical structure recognition* (OCSR) reads structures from images [10]. Rule-based systems such as OSRA [11] were followed by deep-learning models such as MolScribe [12] and DECIMER [13, 14]; DECIMER now includes a model trained for hand-drawn structures [15]. OCSR works on pixels after the drawing is finished, recognises molecules rather than mechanisms, and requires a deep-learning runtime, usually on a server; a benchmark for mechanistic images has only recently been proposed [16]. *Online sketch recognition*, in contrast, interprets the pen strokes as they are drawn. ChemInk [17] showed that this information — stroke order, direction and shape — allows accurate, real-time recognition of hand-drawn molecules on a pen computer, but it handled structures only and is not available as software.

Here we describe MecHand, which brings online recognition of complete mechanisms to the browser. Its design goals were: (i) structures, charges *and* electron-flow

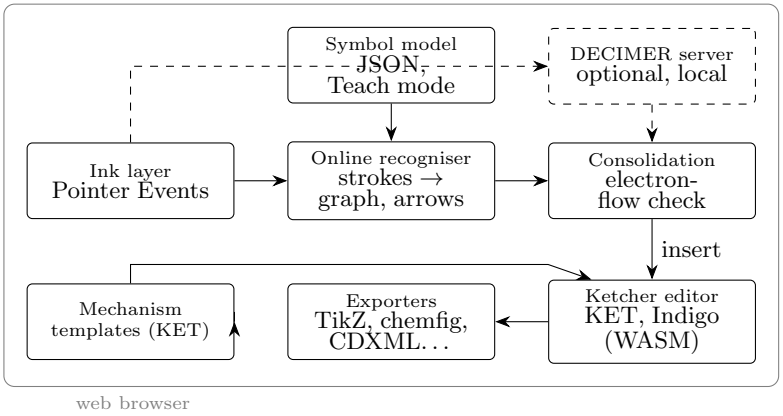


Fig. 1 Architecture of MecHand. Everything inside the grey frame runs in the user's web browser; the DECIMER server (dashed) is optional and runs on the user's own computer.

arrows are recognised together, as drawn on paper; (ii) recognition runs in the browser, with no installation, server or transfer of the user's drawings; (iii) the recognised mechanism lands in a full-featured, standard chemical editor rather than in a separate format; (iv) mechanisms can be exported to LaTeX as editable code; and (v) the recogniser is transparent and its symbol model can be extended by its users.

2 Implementation

2.1 Architecture

MecHand is a single-page web application written in TypeScript with React and built with Vite (Figure 1). Ketcher (version 3.18.0) is used through its published npm packages, pinned to an exact version, rather than as a fork; its standalone mode runs the Indigo toolkit [18] compiled to WebAssembly, so that chemistry operations such as layout, aromaticity and format conversion happen in the browser. MecHand adds an ink layer over Ketcher's drawing area, the online recogniser, a consolidation dialog, mechanism templates and exporters. An optional server wraps DECIMER for users who want to compare the in-browser recogniser with an image-based model.

2.2 Pen input

Strokes are captured with the W3C Pointer Events API, which gives a single code path for mouse, trackpad, touch and stylus. When the browser provides them, coalesced events are used to obtain all samples between screen frames, which matters for fast pen movements. Palm rejection depends on an input mode: by default, once a stylus has been used, or merely hovers over the screen, touch input is ignored for drawing; the user can instead let only the stylus draw, or let fingers always draw. Large contacts and a second simultaneous finger are always ignored. Because drawing with a laptop trackpad normally requires click-and-drag, a trackpad mode draws by click, move without pressing, click. A pen toolbar offers a stroke eraser (also the eraser end of

a stylus), undo and redo of strokes, erasures and clears, and the colour, kind of line (including a pressure- or speed-dependent fountain pen) and width of the ink; the appearance of the ink never affects recognition, which uses the sampled points only. The ink layer disables the browser's touch gestures while freehand mode is on. Turning freehand mode off lets the pointer through to Ketcher, so that hand drawing and Ketcher's tools can be alternated at any time.

2.3 Online recognition

The recogniser turns a list of strokes into a *scene*: atoms (element, formal charge, radical), bonds (order, wedges), curved arrows (start, end and signed bulge) and reaction arrows and plus signs. It uses only geometry and a small symbol classifier, so every decision can be inspected. All thresholds are expressed relative to the drawn bond length L, which makes recognition independent of the size of the drawing (Table 1).

Scale.

L is estimated from two cues: the length of straight stroke segments, and the height h of strokes that are clearly letters (curly and compact, such as C, O, S or B), using $L \approx h/0.38$. When there are few bonds — for example a Cl_2 molecule surrounded by labels — straight letter strokes (the stems of l, H or +) would otherwise bias the first cue, so the letter-based estimate prevails unless both agree. Arrows are detected with a first estimate and excluded before L is refined, because a long reaction arrow would otherwise be taken as a bond.

Arrows.

A stroke is an arrow if it ends with a head: either a sharp reversal at its end, drawn without lifting the pen, or a separate small "V" (two barbs) or single barb whose vertex lies at one end of the stroke and whose arms point back along the shaft. The shaft must be at least $0.75\,L$ long and *smooth*: letters such as N or Z, or a zig-zag chain with a stroke next to its end, have sharp corners that a curved arrow does not. Curved shafts become electron-flow arrows — a full head for an electron pair, a single barb for one electron — and long straight ones with a full head become reaction arrows. Two antiparallel straight shafts side by side, each with a single barb, form an equilibrium sign. The bulge of a curved arrow is the signed maximum distance of the shaft from its chord.

Heads that these rules miss — a closed triangle, a head drawn as two separate barbs, a filled head — are read by an *arrowhead model*: the small strokes that touch the end of a candidate shaft and lie behind it are turned into the frame of the shaft, with the tip at the origin and the shaft along the x axis, and classified by a second $P recogniser into a full head, a half head, or neither. Because the shaft direction at a hand-drawn tip is uncertain, the frame is turned by up to $\pm 30°$ and the best match is kept; a match is accepted above a score of 0.78. A full head read this way also replaces a single barb found by the rules, which is often a stray mark (the stroke of a nearby charge sign) rather than the head.

Table 1 Main recognition parameters, relative to the drawn bond length L.

Parameter	Value
Capital letter height	$0.38\,L$ (scale cue)
Minimum bond piece	$0.4\,L$
Bond ends merged into one atom	within $0.32\,L$
Double/triple bond: angle, separation	$< 15°$, 0.04–$0.4\,L$
Strokes grouped into one label	gap $< 0.22\,L$
Label attached to a bond end	within $0.6\,L$
Charge attached to an atom	within $0.85\,L$
Curved-arrow shaft	$\geq 0.75\,L$, no corner $> 65°$
Reaction arrow	$\geq 1.2\,L$, straight, full head
Arrowhead model	score ≥ 0.78, frame turned by $\leq 30°$
$P cloud size; augmented copies per glyph	32 points; 6
Character acceptance	score ≥ 0.6 (≥ 0.5 with a margin of 0.1)

Letters and bonds.

The remaining strokes are split at their corners with ShortStraw [19], so that chains and rings may be drawn in one stroke. Long straight pieces are bonds. Short strokes are letters when they cross another short stroke (+, t), meet it in a T (H, F), meet it at an acute angle (N, M, Z; bonds never meet at less than 60°), or stand upright beside a letter of the same height (the l of Cl). Bonds stop short of an atom label in handwriting, and these rules keep the bond that ends next to a letter separate from the letter. Close, near-parallel lines are merged into double and triple bonds, and bond ends closer than $0.32\,L$ are merged into atoms.

Atom labels and charges.

Letter strokes are grouped into labels and split into characters, trying a few gap tolerances because letter spacing varies between writers. Each character is classified with the $P point-cloud recogniser [20], which compares resampled clouds of 32 points and is therefore independent of the number, order and direction of the strokes. A decoder then chooses, among the best candidates of each character, the best *chemically valid* reading — for instance "Cl" rather than "CI", "Pt2−" or "O−". A label attaches to the nearest bond end, and absorbs every bond end within $0.5\,L$, because the several bonds of a labelled centre (B in BF_3, a metal) need not meet at one point. A charge sign attaches preferentially to the atom it is written above and to the right of.

Electron flow.

Each end of a curved arrow is anchored to the nearest atom or bond, as in Marvin JS [3]. A simple checker then warns when an electron-pair arrow starts at an atom without a lone pair, when an arrow starts or ends on nothing, or when an arrow would give C, N, O or F more than an octet while no bond to that atom is broken in the same step.

2.4 The symbol model

The "model" of the $P classifier is data: a versioned JSON file of glyph skeletons (letters for common elements, the digits 2–4 for charge magnitudes, and + and −, also circled), drawn as lines and arcs in a unit box. At start-up each skeleton is expanded into copies with random slant, aspect ratio, rotation and jitter, which gives about 600 templates and takes a fraction of a second. Two further sources are added: samples contributed by users and kept in the repository, and samples that each user records in the application's *Teach* mode. The model file also holds arrowhead skeletons (open and closed heads of several angles, heads of two strokes, single barbs) for the arrowhead model.

Teach mode covers the other elements of a mechanism as well: curved arrows for one and two electrons, reaction and equilibrium arrows, single, double and triple bonds, a benzene ring, lone pairs and radical dots. These are drawn in a box with a grey reference bond that fixes the scale, and the application shows at once what it reads. The samples then adapt the geometric rules where these have a free parameter: the head of every arrow drawn with a separate head stroke is cut out, turned into the shaft frame and added to the arrowhead model, so that a personal style of head is recognised from then on; and the size of the dots drawn for lone pairs and radicals raises the size up to which a mark counts as a dot (within a bound, and only for roundish marks, so that a small minus sign stays a charge). Bonds have no such parameter; their samples serve as a check of the writer's style and, when shared, as data to improve the rules.

Teach samples are stored only in the browser's local storage and contain just the symbol or element and the (x, y) points of its strokes. A user can export them and propose them for the shared model through a pull request; the model version in use is shown in the application. Updating the model therefore requires no retraining infrastructure: it is edited, tested and released like the code.

2.5 Consolidation

Recognition never changes the drawing silently. The user presses *Recognize* (or the space bar), and the ink is coloured by how each stroke was read — bonds, labels, arrows, or not understood — while a dialog shows a clean rendering of the result, a confidence estimate and any electron-flow warnings (Figure 2). The user then inserts the result into Ketcher, keeps drawing to correct it, or discards the ink. The scene is passed to Ketcher in its native KET format, with curved arrows as Ketcher's elliptical-arc arrows, so the inserted mechanism is an ordinary, editable Ketcher drawing.

2.6 Mechanism templates

MecHand includes thirteen textbook mechanisms (Table 2): six organic (S_N2, E2, nucleophilic addition to a carbonyl group, the first step of aromatic nitration, a Diels–Alder reaction and the homolysis of Cl_2), four inorganic (the NH_3–BF_3 Lewis adduct, associative substitution at square-planar Pt(II), the hydrolysis of $SiCl_4$ and the Brønsted acidity of $[Fe(OH_2)_6]^{3+}$) and three biochemical (acylation by a serine protease, hydride transfer by alcohol dehydrogenase and phosphoryl transfer by a kinase). They are generated reproducibly with RDKit [21] by a script that also checks

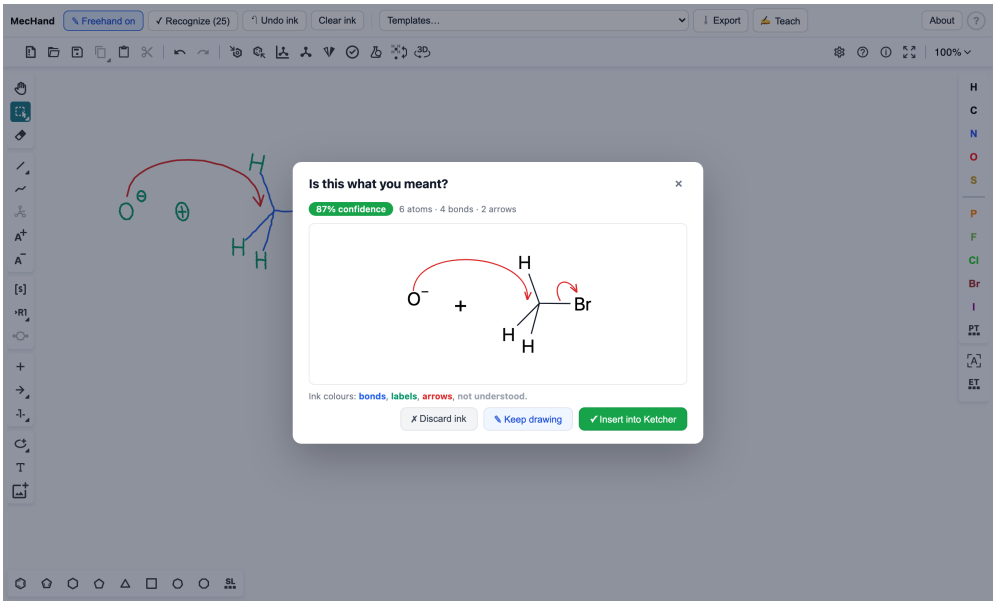


Fig. 2 The consolidation dialog. The S_N2 reactants drawn by hand (left; ink coloured by interpretation) are shown as recognised, with the confidence and a summary, before they are inserted into Ketcher.

that atoms and formal charges balance across every reaction arrow and that every arrow end resolves to the intended atom or bond. They serve as teaching material, as test data for the recogniser and as test cases for the exporters.

2.7 Export

The exporters work on the Ketcher canvas, so they include anything drawn by hand or with Ketcher's tools. The TikZ exporter writes absolute coordinates: atoms with labels become nodes, bonds are drawn between node anchors so that they stop at the labels, wedges become filled or hashed triangles, and Ketcher's elliptical-arc arrows become cubic Bézier curves. For an arrow from $\mathbf{s}$ to $\mathbf{e}$ with bulge h along the unit normal $\mathbf{n}$, the control points

$$\mathbf{c}_1 = \mathbf{s} + \tfrac{4}{3}\,h\,\mathbf{n}, \qquad \mathbf{c}_2 = \mathbf{e} + \tfrac{4}{3}\,h\,\mathbf{n} \tag{1}$$

give a curve whose apex lies at distance h from the chord, as in Ketcher's rendering; fishhooks use a half arrowhead. The `chemfig` exporter writes one `\chemfig` per molecule, built from a spanning tree with absolute bond angles and ring closures, in a `\schemestart`…`\schemestop` reaction scheme, with curved arrows as `\chemmove` commands between named atoms. Figure 3 shows an exported mechanism, compiled as part of this article. CDXML, MOL/RXN, KET and SMILES are produced by Ketcher.

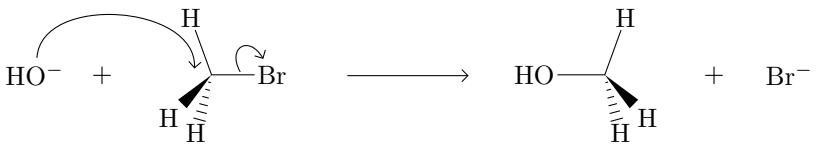


Fig. 3 The S_N2 template exported by MecHand to TikZ and compiled without modification as part of this article.

2.8 Optional recognition server

For comparison with image-based recognition, a small FastAPI service wraps DECIMER (version 2.8.1, hand-drawn model) [15]. MecHand sends it a PNG image of the ink and inserts the structure that DECIMER returns, while arrows and charges still come from the in-browser recogniser. The server is meant to run on the user's own computer and is not needed for normal use.

2.9 Testing, deployment and privacy

The repository includes unit tests of the recogniser, the recognition benchmark described below (whose accuracy thresholds act as a regression test), tests of the KET parser and the exporters, a check that the TikZ and `chemfig` exports of every template compile with pdfLaTeX, and end-to-end tests with Playwright in the Chromium, Firefox and WebKit browser engines. The end-to-end tests open a template, draw a molecule with the pointer, recognise it, insert it into Ketcher and export it. One of them uses every feature of the application and fails if any network request is made after the page has loaded, if any data is sent, or if a cookie is set; MecHand therefore never transmits the user's drawings or handwriting. Continuous integration runs all these tests on every change, and the application and its documentation are deployed as static files to GitHub Pages.

3 Results and discussion

3.1 Recognition of hand-drawn mechanisms

To measure recognition without a user study, each template was rendered as synthetic handwriting in five hand styles, which differ in slant, jitter, letter shapes and in how arrowheads are drawn: with the shaft without lifting the pen, as a separate open "V", as a closed triangle, or as two separate barbs (Figure 4). The last two are read by the arrowhead model. Capital letters were drawn 0.32 bond lengths tall and bonds stopped 0.2 bond lengths short of each label. Each drawing was recognised and compared with the template after aligning the two by scale and translation: an atom is correct if its element and charge match, a bond if it joins the right atoms with the right order, and the arrows if their number and kinds (electron pair, single electron, reaction) match.

Over the 65 drawings, MecHand recovered 86 % of the atoms, 84 % of the bonds and all of the curved and reaction arrows (Table 2); 29 % of the mechanisms were recognised without a single error. Without the arrowhead model, no mechanism drawn with triangle or two-stroke heads had its arrows read correctly (the former were missed,

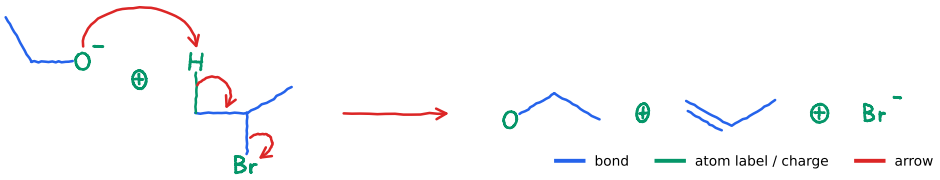


Fig. 4 Synthetic handwriting of the E2 template, with each stroke coloured by how MecHand read it.

Table 2 Recognition of synthetic handwriting of the mechanism templates (five hand styles per template; percentages of atoms, bonds and arrow sets recognised, and of drawings recognised without error).

Template	Class	Atoms	Bonds	Arrows	Exact
SN2	Org.	85	98	100	20
E2	Org.	93	92	100	20
Nucleophilic addition	Org.	88	86	100	0
Electrophilic aromatic substitution	Org.	76	76	100	0
Diels–Alder	Org.	100	100	100	100
Radical homolysis	Org.	100	100	100	100
Lewis acid–base adduct	Inorg.	70	66	100	40
Associative ligand substitution	Inorg.	67	52	100	20
Hydrolysis of $SiCl_4$	Inorg.	98	100	100	60
Brønsted acidity of a metal aqua ion	Inorg.	65	57	100	20
Serine protease acylation	Biochem.	90	92	100	0
Alcohol dehydrogenase	Biochem.	92	91	100	0
Kinase	Biochem.	91	82	100	0
Mean		86	84	100	29

the latter read as single-electron arrows); on the three styles that the rules alone handle, the results with and without it are identical. The Diels–Alder reaction, the homolysis of Cl_2 and the hydrolysis of $SiCl_4$ were recognised completely. The main sources of error were crowded metal complexes, where several labels and charge signs lie close to a metal centre, and labels with many explicit hydrogens.

3.2 Comparison with image-based recognition

To compare with the state of the art for hand-drawn structures, every molecule of the templates (42 molecules, 126 drawings) was drawn in the same three styles and read both by MecHand and by DECIMER's hand-drawn model, which received an image of the same ink. The structures were compared as canonical SMILES with RDKit, after the same normalisation of every side: explicit hydrogens removed, radicals cleared and implicit hydrogens recomputed — so that a label written without its hydrogens is not counted as an error — and stereochemistry ignored. Because DECIMER's accuracy

Table 3 Exact recognition (canonical SMILES) of the 126 drawings of the molecules of the templates by MecHand and by DECIMER 2.8.1 (hand-drawn model).

Templates	Drawings	MecHand	DECIMER
Organic	48	79 %	48 %
Inorganic	27	48 %	30 %
Biochemistry	51	61 %	39 %
All	126	65 %	40 %

depended on the size of the rendered image, it was run at five scales and its best result is reported (21–40 %).

MecHand returned the exact structure for 65 % of the drawings and DECIMER for 40 % (Table 3), with median times of 0.11 s for MecHand in a single JavaScript thread and 0.43 s for DECIMER on a CPU. The comparison is not neutral: the drawings come from MecHand's own synthetic-handwriting generator, which was also used while developing the recogniser, and MecHand uses the order and shape of the strokes, which an image-based model cannot see. It nevertheless shows that, when the strokes are available — as they are while the user draws — a small, transparent recogniser is competitive with a large deep-learning model, without a server and in a fraction of the time.

3.3 Export

The TikZ and `chemfig` exports of all thirteen templates and of twelve test structures compile with pdfLaTeX, which is checked automatically. TikZ output reproduces the layout drawn in Ketcher, including wedges, charges and curved arrows (Figure 3), and is easy to fine-tune by hand because coordinates are absolute. The `chemfig` output follows that package's idioms and suits documents that already use it; its curved arrows are attached to named atoms and require a second compilation.

3.4 Comparison with related software

Table 4 compares MecHand with related tools. MecHand is the only one that combines freehand input, recognition of electron-flow arrows and export to LaTeX; it inherits from Ketcher the editing features of a full structure editor, including stereochemistry, templates, structure checks and format conversion. Marvin JS provides the most complete electron-flow semantics, entered with toolbar tools, and DECIMER the most capable recognition of hand-drawn structures from images.

3.5 Portability and performance

MecHand uses only standard web interfaces (Pointer Events, WebAssembly, Web Workers and SVG) and is tested in the three browser engines that cover current desktop and tablet browsers. According to MDN's compatibility data [22], all of them support these interfaces; Safari added coalesced pointer events only in version 18.2,

Table 4 Features of MecHand and related software (✓: available; –: not available).

	Open source	Runs in browser, no server	Freehand input	Electron-flow arrows	Electron-flow checks	LaTeX export
MecHand	✓	✓	✓	✓	✓	✓
Ketcher [4]	✓	✓	–	✓[a]	–	–
JSME [6]	✓	✓	–	–	–	–
Marvin JS [3]	–	✓	–	✓	✓	–
ChemInk [17]	–	–	✓	–	–	–
DECIMER [15]	✓	–	✓[b]	–	–	–

[a]Elliptical-arc arrows, without attachment to atoms or bonds. [b]Images of finished hand drawings (offline recognition).

so older versions record fewer points per stroke without preventing recognition. The application is about 9 MB compressed, most of it Ketcher's WebAssembly engine, and starts in under two seconds from GitHub Pages on a broadband connection.

3.6 Limitations and future work

The evaluation uses synthetic handwriting. Real handwriting varies more, which is why recognition always ends in a confirmation step and why users can teach MecHand their own writing; a study with students and researchers, recording real pen input, is the next step. Such recordings would also make it possible to add a trained model of stroke sequences, for which the recogniser's scene representation and the optional server already provide the interfaces, and to evaluate on images of mechanisms such as those in SMiCRM [16]. Lone pairs are not yet drawn or exported explicitly, and the electron-flow checker uses simple valence rules rather than full electron counting.

4 Conclusions

MecHand lets chemists and students draw reaction mechanisms by hand, as on paper, and turns them into editable Ketcher drawings and publication-ready LaTeX code. Its online recogniser runs in the browser, keeps every drawing on the user's computer, recognises structures and electron-flow arrows together and checks the electron flow. On synthetic handwriting it recovers most atoms and bonds and all arrows, and it recognises hand-drawn molecules more accurately and faster than an image-based deep-learning model. Because it is built on a standard editor, uses a small and extensible symbol model, and is tested across browsers, MecHand is easy to adopt in teaching and research and to extend.

5 Availability and requirements

- **Project name:** MecHand
- **Project home page:** https://compbiochbiophlab.github.io/MecHand/ (application: https://compbiochbiophlab.github.io/MecHand/app/; source code: https://github.com/CompBiochBiophLab/MecHand)

- **Operating system(s):** Platform independent (web application)
- **Programming language:** TypeScript; Python for the optional recognition server and the analysis scripts
- **Other requirements:** A current web browser (Chrome, Edge, Firefox or Safari); Node.js $\geq$ 20.19 to build it; optional Python 3.10–3.13 for the DECIMER server
- **License:** MIT (MecHand); Apache 2.0 (Ketcher, used as a dependency)
- **Any restrictions to use by non-academics:** None

Abbreviations

CDXML: ChemDraw XML; KET: Ketcher JSON format; MDL: MDL molfile and reaction file formats; OCSR: optical chemical structure recognition; SMILES: simplified molecular input line entry system; WASM: WebAssembly.

Availability of data and materials. The source code, the mechanism templates, the synthetic-handwriting benchmark, the comparison dataset and the scripts that produce every table and figure of this article are available at https://github.com/CompBiochBiophLab/MecHand (`manuscript/analysis/`).

Competing interests. The author declares that he has no competing interests.

Funding. Not applicable.

Authors' contributions. JVF conceived and developed MecHand, performed the analyses and wrote the manuscript.

Acknowledgements. The author thanks the developers of Ketcher, Indigo, RDKit and DECIMER for making their software openly available.

Use of generative AI. MecHand was developed as open-source software on top of open-source components. Generative artificial intelligence (AI) tools based on large language models were used intensively to assist in writing the source code. The conception and design of the approach, and the testing of the software, were carried out by the author. The manuscript was written by the author, with the assistance of generative AI in drafting the technical descriptions of the software. The author reviewed and edited all code and text, worked on the final version of the manuscript, and takes full responsibility for the content of this publication.

References

[1] Bhattacharyya G, Bodner GM. "It Gets Me to the Product": How Students Propose Organic Mechanisms. Journal of Chemical Education. 2005;82:1402. https://doi.org/10.1021/ed082p1402.

[2] Grove NP, Cooper MM, Rush KM. Decorating with Arrows: Toward the Development of Representational Competence in Organic Chemistry. Journal of Chemical Education. 2012;89:844–849. https://doi.org/10.1021/ed2003934.

[3] ChemAxon.: Electron flow arrows – Marvin JS documentation. Accessed 9 November 2026. https://docs.chemaxon.com/Electron_Flow_Arrows.html.

[4] Karulin B, Kozhevnikov M. Ketcher: web-based chemical structure editor. Journal of Cheminformatics. 2011;3(Suppl 1):P3. https://doi.org/10.1186/1758-2946-3-S1-P3.

[5] EPAM Systems.: Ketcher: A Web-Based Chemical Structure Editor. Available from: https://github.com/epam/ketcher.

[6] Bienfait B, Ertl P. JSME: a free molecule editor in JavaScript. Journal of Cheminformatics. 2013;5:24. https://doi.org/10.1186/1758-2946-5-24.

[7] Jiang C, Jin X, Dong Y, Chen M. Kekule.js: An Open Source JavaScript Chemoinformatics Toolkit. Journal of Chemical Information and Modeling. 2016;56:1132–1138. https://doi.org/10.1021/acs.jcim.6b00167.

[8] Tellechea C.: The `chemfig` package. Available from: https://ctan.org/pkg/chemfig.

[9] Tantau T.: The TikZ and PGF packages. Available from: https://ctan.org/pkg/pgf.

[10] Rajan K, Brinkhaus HO, Zielesny A, Steinbeck C. A review of optical chemical structure recognition tools. Journal of Cheminformatics. 2020;12:60. https://doi.org/10.1186/s13321-020-00465-0.

[11] Filippov IV, Nicklaus MC. Optical Structure Recognition Software To Recover Chemical Information: OSRA, An Open Source Solution. Journal of Chemical Information and Modeling. 2009;49:740–743. https://doi.org/10.1021/ci800067r.

[12] Qian Y, Guo J, Tu Z, et al. MolScribe: Robust Molecular Structure Recognition with Image-to-Graph Generation. Journal of Chemical Information and Modeling. 2023;63:4321–4329. https://doi.org/10.1021/acs.jcim.2c01480.

[13] Rajan K, Zielesny A, Steinbeck C. DECIMER 2.0: Deep Learning for Chemical Image Recognition using Transformers. Journal of Cheminformatics. 2023;15:17. https://doi.org/10.1186/s13321-023-00703-1.

[14] Rajan K, Brinkhaus HO, Agea MI, Zielesny A, Steinbeck C. DECIMER.ai: an open platform for automated optical chemical structure identification, segmentation and recognition in scientific publications. Nature Communications. 2023;14:5045. https://doi.org/10.1038/s41467-023-40782-0.

[15] Brinkhaus HO, Rajan K, Zielesny A, Steinbeck C. Advancements in hand-drawn chemical structure recognition through an enhanced DECIMER architecture. Journal of Cheminformatics. 2024;16:78. https://doi.org/10.1186/

s13321-024-00872-7.

[16] Leung CT, Chen Y, Gao H.: SMiCRM: A Benchmark Dataset of Mechanistic Molecular Images. arXiv:2407.18338.

[17] Ouyang TY, Davis R. ChemInk: a natural real-time recognition system for chemical drawings. In: Proceedings of the 16th International Conference on Intelligent User Interfaces (IUI '11); 2011. p. 267–276.

[18] Pavlov D, Rybalkin M, Karulin B, Kozhevnikov M, Savelyev A, Churinov A. Indigo: universal cheminformatics API. Journal of Cheminformatics. 2011;3(Suppl 1):P4. https://doi.org/10.1186/1758-2946-3-S1-P4.

[19] Wolin A, Eoff B, Hammond T. ShortStraw: A Simple and Effective Corner Finder for Polylines. In: Eurographics Workshop on Sketch-Based Interfaces and Modeling (SBIM); 2008. p. 33–40.

[20] Vatavu RD, Anthony L, Wobbrock JO. Gestures as point clouds: a $P recognizer for user interface prototypes. In: Proceedings of the 14th ACM International Conference on Multimodal Interaction (ICMI '12); 2012. p. 273–280.

[21] RDKit contributors.: RDKit: Open-source cheminformatics. Version 2026.09. https://www.rdkit.org.

[22] MDN contributors.: Browser compatibility data (@mdn/browser-compat-data), version 8.1.8. https://github.com/mdn/browser-compat-data.